

in between  $-f_c$  and  $f_c$ . Then we look to the Fourier transform to tell whether the continuous function *has* been competently sampled (aliasing effects minimized). We do this by looking to see whether the Fourier transform is already approaching zero as the frequency approaches  $f_c$  from below or  $-f_c$  from above. If, on the contrary, the transform is going toward some finite value, then chances are that components outside of the range have been folded back over onto the critical range.

### 12.1.2 Discrete Fourier Transform

We now estimate the Fourier transform of a function from a finite number of its sampled points. Suppose that we have  $N$  consecutive sampled values,

$$h_k \equiv h(t_k), \quad t_k \equiv k\Delta, \quad k = 0, 1, 2, \dots, N-1 \quad (12.1.4)$$

so that the sampling interval is  $\Delta$ . To make things simpler, let us also suppose that  $N$  is even. If the function  $h(t)$  is nonzero only in a finite interval of time, then that whole interval of time is supposed to be contained in the range of the  $N$  points given. Alternatively, if the function  $h(t)$  goes on forever, then the sampled points are supposed to be at least “typical” of what  $h(t)$  looks like at all other times.

With  $N$  numbers of input, we will evidently be able to produce no more than  $N$  independent numbers of output. So, instead of trying to estimate the Fourier transform  $H(f)$  at all values of  $f$  in the range  $-f_c$  to  $f_c$ , let us seek estimates only at the discrete values

$$f_n \equiv \frac{n}{N\Delta}, \quad n = -\frac{N}{2}, \dots, \frac{N}{2} \quad (12.1.5)$$

The extreme values of  $n$  in (12.1.5) correspond exactly to the lower and upper limits of the Nyquist critical frequency range. If you are really on the ball, you will have noticed that there are  $N+1$ , not  $N$ , values of  $n$  in (12.1.5); it will turn out that the two extreme values of  $n$  are not independent (in fact they are equal), but all the others are. This reduces the count to  $N$ .

The remaining step is to approximate the integral in (12.0.1) by a discrete sum:

$$H(f_n) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f_n t} dt \approx \sum_{k=0}^{N-1} h_k e^{2\pi i f_n t_k} \Delta = \Delta \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N} \quad (12.1.6)$$

Here equations (12.1.4) and (12.1.5) have been used in the final equality. The final summation in equation (12.1.6) is called the *discrete Fourier transform* of the  $N$  points  $h_k$ . Let us denote it by  $H_n$ ,

$$H_n \equiv \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N} \quad (12.1.7)$$

The discrete Fourier transform maps  $N$  complex numbers (the  $h_k$ 's) into  $N$  complex numbers (the  $H_n$ 's). It does not depend on any dimensional parameter, such as the time scale  $\Delta$ . The relation (12.1.6) between the discrete Fourier transform of a set of numbers and their continuous Fourier transform when they are viewed as samples of a continuous function sampled at an interval  $\Delta$  can be rewritten as

$$H(f_n) \approx \Delta H_n \quad (12.1.8)$$

where  $f_n$  is given by (12.1.5).

Up to now we have taken the view that the index  $n$  in (12.1.7) varies from  $-N/2$  to  $N/2$  (cf. 12.1.5). You can easily see, however, that (12.1.7) is periodic in  $n$ , with period  $N$ . Therefore,  $H_{-n} = H_{N-n}$ ,  $n = 1, 2, \dots$ . With this conversion in mind, one generally lets the  $n$  in  $H_n$  vary from 0 to  $N - 1$  (one complete period). Then  $n$  and  $k$  (in  $h_k$ ) vary exactly over the same range, so the mapping of  $N$  numbers into  $N$  numbers is manifest. When this convention is followed, you must remember that zero frequency corresponds to  $n = 0$  and positive frequencies  $0 < f < f_c$  correspond to values  $1 \leq n \leq N/2 - 1$ , while negative frequencies  $-f_c < f < 0$  correspond to  $N/2 + 1 \leq n \leq N - 1$ . The value  $n = N/2$  corresponds to *both*  $f = f_c$  and  $f = -f_c$ .

The discrete Fourier transform has symmetry properties almost exactly the same as the continuous Fourier transform. For example, all the symmetries in the table following equation (12.0.3) hold if we read  $h_k$  for  $h(t)$ ,  $H_n$  for  $H(f)$ , and  $H_{N-n}$  for  $H(-f)$ . (Likewise, “even” and “odd” in time refer to whether the values  $h_k$  at  $k$  and  $N - k$  are identical or the negative of each other.)

The formula for the discrete *inverse* Fourier transform, which recovers the set of  $h_k$ ’s exactly from the  $H_n$ ’s is

$$h_k = \frac{1}{N} \sum_{n=0}^{N-1} H_n e^{-2\pi i k n / N} \quad (12.1.9)$$

Notice that the only differences between (12.1.9) and (12.1.7) are (i) changing the sign in the exponential, and (ii) dividing the answer by  $N$ . This means that a routine for calculating discrete Fourier transforms can also, with slight modification, calculate the inverse transforms.

The discrete form of Parseval’s theorem is

$$\sum_{k=0}^{N-1} |h_k|^2 = \frac{1}{N} \sum_{n=0}^{N-1} |H_n|^2 \quad (12.1.10)$$

There are also discrete analogs to the convolution and correlation theorems (equations 12.0.10 and 12.0.12), but we shall defer them to §13.1 and §13.2, respectively.

#### CITED REFERENCES AND FURTHER READING:

- Brigham, E.O. 1974, *The Fast Fourier Transform* (Englewood Cliffs, NJ: Prentice-Hall).  
 James, J.F. 2002, *A Student’s Guide to Fourier Transforms*, 2nd ed. (Cambridge, UK: Cambridge University Press)  
 Elliott, D.F., and Rao, K.R. 1982, *Fast Transforms: Algorithms, Analyses, Applications* (New York: Academic Press).

## 12.2 Fast Fourier Transform (FFT)

How much computation is involved in computing the discrete Fourier transform (12.1.7) of  $N$  points? For many years, until the mid-1960s, the standard answer was this: Define  $W$  as the complex number

$$W \equiv e^{2\pi i / N} \quad (12.2.1)$$

Then (12.1.7) can be written as

$$H_n = \sum_{k=0}^{N-1} W^{nk} h_k \quad (12.2.2)$$

In other words, the vector of  $h_k$ 's is multiplied by a matrix whose  $(n, k)$ th element is the constant  $W$  to the power  $n \times k$ . The matrix multiplication produces a vector result whose components are the  $H_n$ 's. This matrix multiplication evidently requires  $N^2$  complex multiplications, plus a smaller number of operations to generate the required powers of  $W$ . So, the discrete Fourier transform appears to be an  $O(N^2)$  process. These appearances are deceiving! The discrete Fourier transform can, in fact, be computed in  $O(N \log_2 N)$  operations with an algorithm called the *fast Fourier transform*, or *FFT*. The difference between  $N \log_2 N$  and  $N^2$  is immense. With  $N = 10^8$ , for example, it is a factor of several million, comparable to the ratio of one second to one month. The existence of an FFT algorithm became generally known only in the mid-1960s, from the work of J.W. Cooley and J.W. Tukey. Retrospectively, we now know (see [1]) that efficient methods for computing the DFT had been independently discovered, and in some cases implemented, by as many as a dozen individuals, starting with Gauss in 1805!

One "rediscovery" of the FFT, that of Danielson and Lanczos in 1942, provides one of the clearest derivations of the algorithm. Danielson and Lanczos showed that a discrete Fourier transform of length  $N$  can be rewritten as the sum of two discrete Fourier transforms, each of length  $N/2$ . One of the two is formed from the even-numbered points of the original  $N$ , the other from the odd-numbered points. The proof is simply this:

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} e^{2\pi i j k / N} f_j \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k (2j) / N} f_{2j} + \sum_{j=0}^{N/2-1} e^{2\pi i k (2j+1) / N} f_{2j+1} \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j} + W^k \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j+1} \\ &= F_k^e + W^k F_k^o \end{aligned} \quad (12.2.3)$$

In the last line,  $W$  is the same complex constant as in (12.2.1),  $F_k^e$  denotes the  $k$ th component of the Fourier transform of length  $N/2$  formed from the even components of the original  $f_j$ 's, while  $F_k^o$  is the corresponding transform of length  $N/2$  formed from the odd components. Notice also that  $k$  in the last line of (12.2.3) varies from 0 to  $N$ , not just to  $N/2$ . Nevertheless, the transforms  $F_k^e$  and  $F_k^o$  are periodic in  $k$  with length  $N/2$ . So each is repeated through two cycles to obtain  $F_k$ .

The wonderful thing about the *Danielson-Lanczos lemma* is that it can be used recursively. Having reduced the problem of computing  $F_k$  to that of computing  $F_k^e$  and  $F_k^o$ , we can do the same reduction of  $F_k^e$  to the problem of computing the transform of its  $N/4$  even-numbered input data and  $N/4$  odd-numbered data. In other words, we can define  $F_k^{ee}$  and  $F_k^{eo}$  to be the discrete Fourier transforms of the

points that are respectively even-even and even-odd on the successive subdivisions of the data.

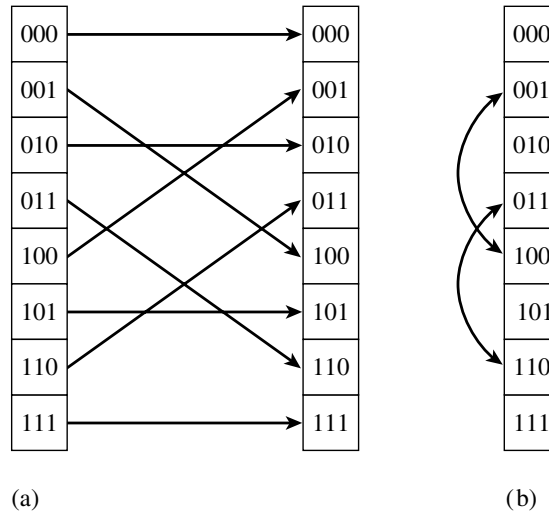
Although there are ways of treating other cases, by far the easiest case is the one in which the original  $N$  is an integer power of 2. In fact, we categorically recommend that you *only* use FFTs with  $N$  a power of 2. If the length of your data set is not a power of 2, pad it with zeros up to the next power of 2. (We will give more sophisticated suggestions in subsequent sections below.) With this restriction on  $N$ , it is evident that we can continue applying the Danielson-Lanczos lemma until we have subdivided the data all the way down to transforms of length one. What is the Fourier transform of length one? It is just the identity operation that copies its one input number into its one output slot! In other words, for every pattern of  $\log_2 N$   $e$ 's and  $o$ 's, there is a one-point transform that is just one of the input numbers  $f_n$ ,

$$F_k^{eooooo\cdots oee} = f_n \quad \text{for some } n \quad (12.2.4)$$

(Of course this one-point transform actually does not depend on  $k$ , since it is periodic in  $k$  with period 1.)

The next trick is to figure out which value of  $n$  corresponds to which pattern of  $e$ 's and  $o$ 's in equation (12.2.4). The answer is: Reverse the pattern of  $e$ 's and  $o$ 's, then let  $e = 0$  and  $o = 1$ , and you will have, *in binary*, the value of  $n$ . Do you see why it works? It is because the successive subdivisions of the data into even and odd are tests of successive low-order (least significant) bits of  $n$ . This idea of *bit reversal* can be exploited in a very clever way that, along with the Danielson-Lanczos lemma, makes FFTs practical: Suppose we take the original vector of data  $f_j$  and rearrange it into bit-reversed order (see Figure 12.2.1), so that the individual numbers are in the order not of  $j$ , but of the number obtained by bit reversing  $j$ . Then the bookkeeping on the recursive application of the Danielson-Lanczos lemma becomes extraordinarily simple. The points as given are the one-point transforms. We combine adjacent pairs to get two-point transforms, then combine adjacent pairs of pairs to get four-point transforms, and so on, until the first and second halves of the whole data set are combined into the final transform. Each combination takes of order  $N$  operations, and there are evidently  $\log_2 N$  combinations, so the whole algorithm is of order  $N \log_2 N$  (assuming, as is the case, that the process of sorting into bit-reversed order is no greater in order than  $N \log_2 N$ ).

This, then, is the structure of an FFT algorithm: It has two sections. The first section sorts the data into bit-reversed order. Luckily this takes no additional storage, since it involves only swapping pairs of elements. (If  $k_1$  is the bit reverse of  $k_2$ , then  $k_2$  is the bit reverse of  $k_1$ .) The second section has an outer loop that is executed  $\log_2 N$  times and calculates, in turn, transforms of length 2, 4, 8,  $\dots$ ,  $N$ . This series of operations is often called a *butterfly*. For each stage of the process, two nested inner loops range over the subtransforms already computed and the elements of each transform, implementing the Danielson-Lanczos lemma. The operation is made more efficient by restricting external calls for trigonometric sines and cosines to the outer loop, where they are made only  $\log_2 N$  times. Computation of the sines and cosines of multiple angles is through simple recurrence relations in the inner loops (cf. 5.4.6).



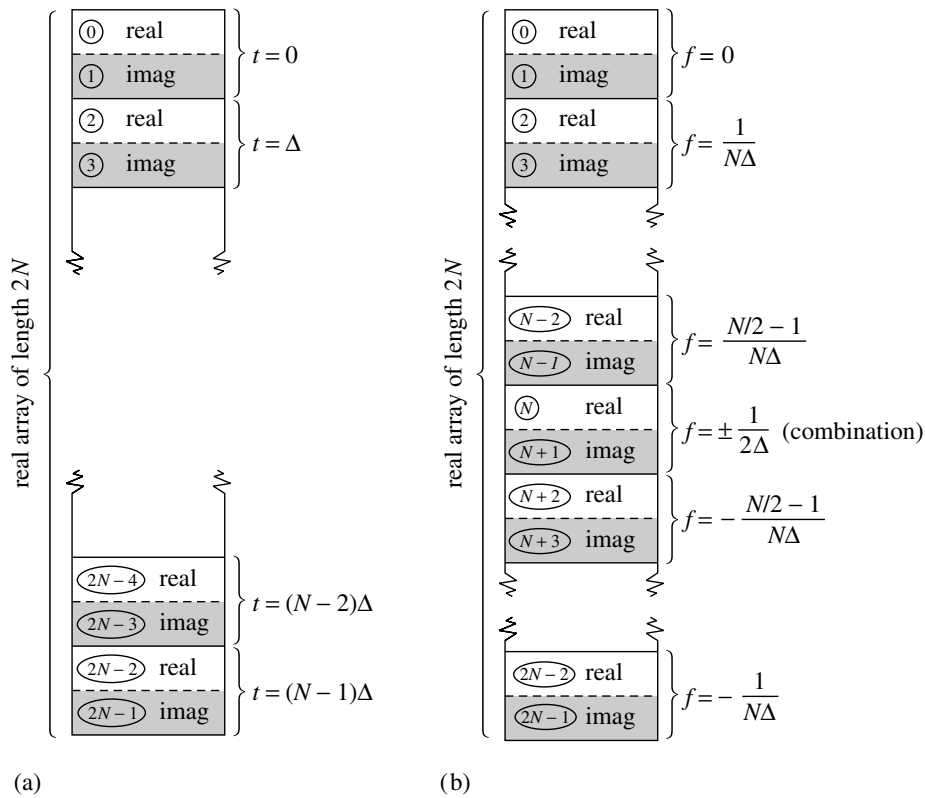
**Figure 12.2.1.** Reordering an array (here of length 8) by bit reversal, (a) between two arrays, versus (b) in place. Bit-reversal reordering is a necessary part of the fast Fourier transform (FFT) algorithm.

### 12.2.1 Bare FFT Routine and Helper Interfaces

Experience convinces us that a good way to package the FFT is as (i) a bare routine with a minimal interface, plus also (ii) a small set of interface objects that make it easier to get data in and out of the bare routine. The bare FFT routine given below is based on one originally written by N.M. Brenner. The input quantities are the number of complex data points  $n$  ( $=N$ ), a pointer to the data array (`data[0..2*n-1]`), and `isign`, which is set to either  $\pm 1$  and is the sign of  $i$  in the exponential of equation (12.1.7). When `isign` is set to  $-1$ , the routine thus calculates the inverse transform (12.1.9) — except that it does not multiply by the normalizing factor  $1/N$  that appears in that equation. You do that yourself. We test to be sure that  $n$  is a power of 2 by the C++ idiom `n&(n-1)`, which is zero only if  $n$  is, in binary, 1 followed by any number of zeros.

Notice that the argument  $n$  is the number of *complex* data points. The actual length of the Doub array (`data[0..2*n-1]`) is  $2n$ , with each complex value occupying two consecutive locations. In other words, `data[0]` is the real part of  $f_0$ , `data[1]` is the imaginary part of  $f_0$ , and so on up to `data[2*n-2]`, which is the real part of  $f_{N-1}$ , and `data[2*n-1]`, which is the imaginary part of  $f_{N-1}$ .

The FFT routine gives back the  $F_n$ 's packed in exactly the same fashion, as  $n$  complex numbers. The real and imaginary parts of the zero frequency component  $F_0$  are in `data[0]` and `data[1]`; the smallest nonzero positive frequency has real and imaginary parts in `data[2]` and `data[3]`; the smallest (in magnitude) nonzero negative frequency has real and imaginary parts in `data[2*n-2]` and `data[2*n-1]`. Positive frequencies increasing in magnitude are stored in the real-imaginary pairs `data[4]`, `data[5]` up to `data[n-2]`, `data[n-1]`. Negative frequencies of increasing magnitude are stored in `data[2*n-4]`, `data[2*n-3]` down to `data[n+2]`, `data[n+3]`. Finally, the pair `data[n]`, `data[n+1]` contains the real and imaginary parts of the one aliased point that contains the most positive and the most negative frequencies. You should try to develop a familiarity with this storage arrangement of



**Figure 12.2.2.** Input and output arrays for FFT. (a) The input array contains  $N$  (a power of 2) complex time samples in a real array of length  $2N$ , with real and imaginary parts alternating. (b) The output array contains the complex Fourier spectrum at  $N$  values of frequency. Real and imaginary parts again alternate. The array starts with zero frequency, works up to the most positive frequency (which is aliased with the most negative frequency). Negative frequencies follow, from the second-most negative up to the frequency just below zero.

complex spectra, also shown in Figure 12.2.2, since it is the practical standard.

```
fourier.h void four1(Doub *data, const Int n, const Int isign) {
    Replaces data[0..2*n-1] by its discrete Fourier transform, if isign is input as 1; or replaces
    data[0..2*n-1] by n times its inverse discrete Fourier transform, if isign is input as -1. data
    is a complex array of length n stored as a real array of length 2*n. n must be an integer power
    of 2.
    Int nn, mmax, m, j, istep, i;
    Doub wtemp, wr, wpr, wpi, wi, theta, tempr, tempi;
    if (n<2 || n&(n-1)) throw("n must be power of 2 in four1");
    nn = n << 1;
    j = 1;
    for (i=1; i<nn; i+=2) {
        if (j > i) {
            SWAP(data[j-1], data[i-1]);
            SWAP(data[j], data[i]);
        }
        m=n;
        while (m >= 2 && j > m) {
            j -= m;

```

This is the bit-reversal section of the routine.  
Exchange the two complex numbers.